LOCUS: Verifying OCR by Local Optimality over Counterfactual String Graphs

Shikhar Dubey[1,*,†], **Sanhita Saxena**[2,*]

[1]Intelligent Systems, School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, Himachal Pradesh, India
[2]School of Mathematical and Statistical Sciences, Indian Institute of Technology Mandi, Himachal Pradesh, India
[*]These authors contributed equally to this work. [†]Corresponding author.

E-mail: [1]s24130@students.iitmandi.ac.in, [2]s25008@students.iitmandi.ac.in

Abstract

Optical character recognition engines always return a transcription, so every pipeline must decide, word by word, whether to trust it. Existing verifiers judge the claim in isolation, by engine confidence, by agreement with a second recogniser, or by asking a large vision-language model. We instead ask whether the claim is the best explanation of the image among the strings it could have been confused with. LOCUS builds around each claim a small graph of plausible single-character edits and independent readings, scores every candidate string by how much better it explains the real image than a clean rendering of itself, cancelling the recogniser's language prior, and tests whether the claim is locally optimal with a relation-aware graph neural network. The network is pretrained without human labels from engine agreement and synthetic corruptions and adapts from a few labelled words. We introduce a benchmark of real recognition errors from two engines on five public datasets covering forms, receipts, handwriting and scene text. With 25 labelled words per class, LOCUS ranks correct above incorrect transcriptions better than engine confidence, learned confidence, engine voting, Florence-2 re-recognition and a seven-billion-parameter vision-language judge on every dataset, and attains the best average macro-F1. It is significantly better than re-recognition with a strong recogniser where that recogniser is fallible, comparable where it is in-domain, and behind it on its own scene-text benchmark. Controlled analyses show that the gain comes from the explicit counterfactual evidence and label-free pretraining: simpler learners on the same evidence come within 0.015 macro-F1 of the graph network, and message passing is not needed. LOCUS also rejects most errors whose true string lies outside its graph. A cascade that runs LOCUS only on words a fast recogniser does not confidently confirm keeps its macro-F1 at 1.3–2.6 times the throughput.

Keywords: OCR verification; counterfactual reasoning; graph neural networks; few-shot learning; render normalisation; document analysis

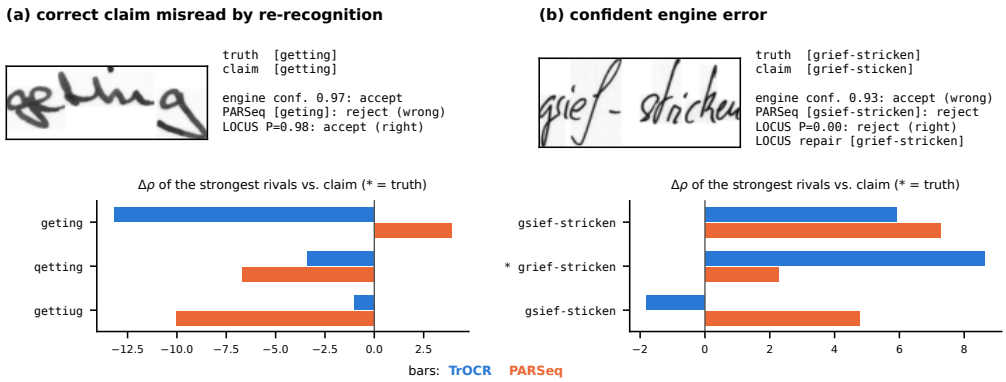


Figure 1. Why compare hypotheses instead of scoring one. Two real IAM test words. (a) A correct claim that PARSeq misreads, so re-recognition rejects it; PARSeq prefers its own reading (orange bar) but TrOCR strongly disfavours it (blue bar), and LOCUS, which weighs both verifiers' evidence for every rival, accepts the claim (this happens for 312 of the 419 correct IAM claims that PARSeq misreads). (b) A confident engine error that engine confidence accepts; LOCUS rejects it and proposes the correct string. Bars: render-normalised evidence of the strongest rivals relative to the claim ($*$: ground truth).

1. Introduction

Optical character recognition (OCR) engines always return a transcription, and modern end-to-end and vision–language readers return fluent text even when they are wrong [1, 2]. Every consumer of OCR (key-information extraction from receipts and forms, archival search, accessibility, data entry) must therefore decide, word by word, whether a transcription can be trusted or must be routed to a human. This is *OCR verification*: given a word image x and a claim h produced by an arbitrary, possibly black-box engine, predict whether h is exactly what is written in x. The three families of verifiers used in practice each have a characteristic blind spot (Figure 1). **Engine confidence** is unavailable for many engines and overconfident on fluent, plausible errors. **Re-recognition and voting** (accept h iff a second recogniser reads the same string [3–5]) is a binary test that inherits every bias of the second recogniser. **Vision–language judges** [6] are billions of parameters large, cannot be adapted from a handful of labels and, as we show, are poor at the character-exact judgements verification needs.

Key idea. The right question is not "does the image look like h?" but "is h the best explanation of the image among the strings it could have been confused with?" An OCR error is a near miss: a confusable substitution (`O`/`0`, `1`/`l`), a dropped or inserted glyph, or a case flip. We build around every claim a small *counterfactual string graph* of its OCR-plausible edits and test whether the claim is a *local optimum* of visual evidence on it. Two problems must be solved. First, the score must be *visual*: a recogniser's likelihood $s(v \mid x)$ mixes how well v explains the pixels with how plausible v is as a string, so rare but correct strings are penalised. We cancel this prior with a *render-normalised likelihood ratio* $\rho(v) = s(v \mid x) - s(v \mid R(v))$, where $R(v)$ is a clean rendering of v. Second, the comparison must be learned from very few labels: a relation-aware graph network with an explicit local-optimality readout is pretrained *without human labels* on images where two engines agree, paired with synthetic corruptions whose repair is known.

Contributions. (1) A formulation of OCR verification as testing the local optimality of a claim among OCR-plausible counterfactuals, which also yields a repair proposal (Section 3). (2) Render-normalised counterfactual evidence that removes the recogniser's string prior, computed with two frozen, architecturally different recognisers. This evidence is the principal contribution: our analyses show that it, together with label-free pretraining, accounts for the gain. (3) LOCUS, a relation-aware GNN over counterfactual string graphs, pretrained label-free and adapted from $k \in \{5, \ldots, 100\}$ labelled words per class. The graph network is a convenient, permutation-invariant learner for this evidence that supports label-free pretraining with a repair objective; we show explicitly that it is not necessary, since simpler learners on the same evidence perform almost as well (Sections 5.3 and 6.3). (4) A verification benchmark of *real* OCR errors on five public datasets (forms, camera and scanned receipts, handwriting, scene text) with two engines, and a comparison with engine and learned confidence, voting, three re-recognition baselines and a 7B vision–language judge (Sections 4–5). (5) An honest analysis: review cost, transfer and recalibration, repair, document-level significance with a design-effect correction, recogniser dependence including correlated recogniser failures, graph coverage, graph capacity, accuracy–throughput trade-offs, pseudo-label noise, rendering sensitivity, message passing, and a manual audit with representative qualitative examples (Sections 6–7).

2. Related Work

OCR confidence and failure prediction. Engine confidence is the default verifier, and learned failure predictors regress the true-class probability from a classifier's own features [7]. Both score a single hypothesis in isolation and need access to engine internals; neither compares the claim with its visual competitors.

Post-OCR correction with large models. Since 2024 post-OCR processing has moved to large (multimodal) language models. LLM post-correction gives "no free lunches": gains depend on language, model size and input error rate, and correct text can be damaged [8]. Vision–language models are also used as judges of recognition output, and fine-grained OCR benchmarks show that even the strongest multimodal models still make frequent character-level errors [1]. Such systems target page-level reading, need large curated training sets and are orders of magnitude more expensive per word; we include a 7B judge [6] as a baseline.

End-to-end OCR models and their hallucinations. OCR-free document readers such as Donut [9] and VLM-based PDF linearisers such as olmOCR [10] read whole pages. Their errors are often *plausible*: multimodal LLMs hallucinate text on degraded images [2]. This motivates an engine-agnostic verifier that asks whether the image supports *this* string better than its near-miss rivals. Strong recognisers such as TrOCR [4] and PARSeq [3] are natural, frozen verifiers in this setting.

Contrast against a reference. Visual contrastive decoding reduces hallucination by contrasting a model's outputs on the real and on a distorted image [11]. Our render-normalised ratio applies the same principle with a reference *specific to each hypothesis*: a clean rendering of the very string being tested.

Graph networks for OCR verification. Character-level graphs comparing real and synthetic glyphs have been used to verify Hindi OCR [12], and Saxena *et al.* [13] represent each word as a relational line graph of real-versus-rendered character comparisons, verified few-shot by a GNN on a single private low-resolution dataset. Both require character segmentation of the word image and compare characters, not alternative readings. LOCUS needs no segmentation: its nodes are *hypotheses* and its edges are edit operations between them, encoded with relation-aware attention [14].

Pseudo-labels, calibration and label shift. Training on pseudo-labels risks *confirmation bias* [15], commonly countered by confidence thresholding [16]; we test both (Section 6.7). Calibration degrades under distribution shift [17]; a new OCR engine acts like a label shift, for which EM prior re-estimation is a strong label-free correction [18], and conformal prediction offers distribution-free accept/review guarantees [19].

3. Proposed Method: LOCUS

Problem statement. An OCR engine $\mathscr{E}$ maps a word (or text-line) image x to a claim $h = \mathscr{E}(x)$. The engine is a black box: we observe only h. With t the true transcription, verification predicts $y = \mathbf{1}[h = t]$ (CORRECT vs. INCORRECT) and, optionally, proposes a repair $\hat{t}$. We study the label-efficient regime with only $k \in \{5, 10, 25, 50, 100\}$ labelled words per class plus an unlabelled pool of images with engine outputs, and also report full supervision. Figure 2 gives an overview on a real test word and Algorithm 1 summarises inference.

3.1 Counterfactual string graphs

For each claim h we build a graph $G(h) = (\mathscr{V}, \mathscr{E})$ whose nodes are strings: the claim and up to $M = 28$ counterfactual neighbours generated without any ground truth, in this priority order: (i) the greedy readings g_1, g_2 of two frozen verifier recognisers V_1, V_2; (ii) *reading-aligned edits*, the single substitutions, insertions and deletions that move h one step towards g_1 or g_2 under a Levenshtein alignment; (iii) *confusion edits*: substitutions within visually confusable groups (e.g. {O,0,o,Q,D}, {I,l,1,|}, {5,S}), case flips and single deletions. Every neighbour is linked to the claim by an edge typed by its edit operation (seven types); *sibling* edges link neighbours that edit the same position (competing explanations of one glyph) and *path* edges link each reading to the single edits leading to it, giving a one-hot edge type $e_{uv} \in \{0,1\}^{11}$. OCR errors are near misses, so the truth lies inside $G(h)$ for most real errors (Supplementary Table S33), which makes repair possible.

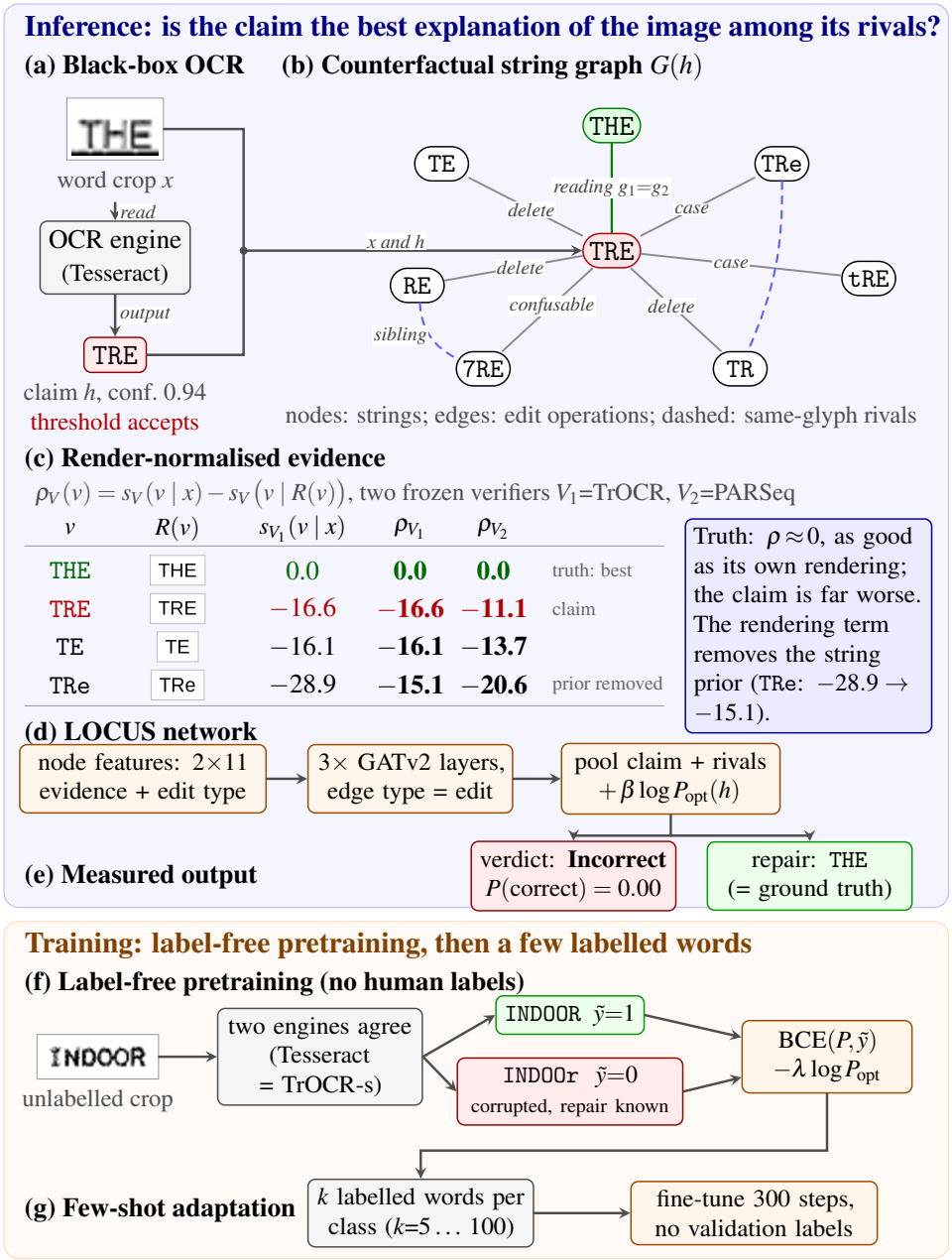
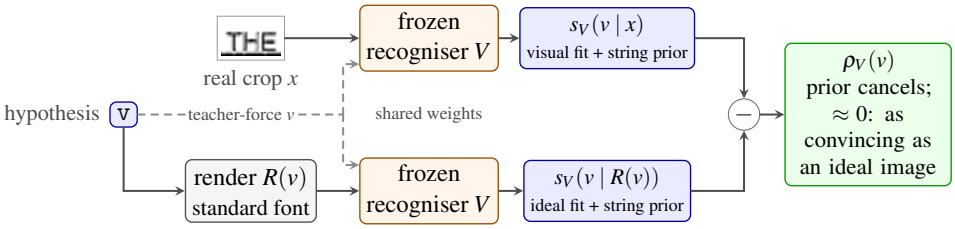


Figure 2. LOCUS at a glance, on a real FUNSD test word (truth THE). (a) A black-box engine reads TRE. (b) LOCUS builds the claim's counterfactual string graph. (c) Every hypothesis is scored by its render-normalised likelihood ratio under two frozen recognisers. (d) A relation-aware graph network compares the claim with its rivals and (e) rejects it, proposing the ground truth as repair. (f) Pretraining uses no human labels; (g) adaptation uses k labelled words per class. All strings, images and numbers shown are real.

3.2 Render-normalised evidence

Let $s_V(v \mid x) = \sum_i \log p_V(v_i \mid v_{<i}, x)$ be the teacher-forced log-likelihood of string v under a frozen autoregressive recogniser V. It conflates visual fit with the prior that V's decoder has learned over strings: a correct but rare string (a serial number, a surname, lower-case text under a recogniser trained on upper-case receipts) scores low. With $R(v)$ a clean rendering of v in a

(a) The ratio for one hypothesis v



(b) Measured on the real word (truth `THE`; TrOCR as V)

v	$R(v)$	$s_V(v \mid x)$	$s_V(v \mid R(v))$	$\rho_V(v)$
THE	THE	0.0	0.0	**0.0**
TRE	TRE	−16.6	0.0	**−16.6**
TRe	TRe	−28.9	−13.8	**−15.1**

`TRE` and `TRe` differ only in case. Raw scores differ by 12.3, mostly string prior (`TRe` scores −13.8 on its own clean rendering); after normalisation they differ by 1.5.

Figure 3. Render normalisation. (a) The same frozen recogniser scores hypothesis v twice, teacher-forced on the real crop and on a clean rendering of v; the difference cancels the decoder's prior over strings. (b) Real TrOCR values for the word of Figure 2: the case variant `TRe` looks far less likely than `TRE` on the raw score only because TrOCR's decoder dislikes the string, which the ratio removes.

standard font at the height of x, we define

$$\rho_V(v) \;=\; s_V(v \mid x) \;-\; s_V\big(v \mid R(v)\big). \tag{1}$$

Under the factorisation $\log p_V(v \mid x) \approx \log p_{\text{vis}}(x \mid v) + \log \pi_V(v) - \log Z(x)$ the string prior $\pi_V(v)$ appears in both terms and cancels; $\rho_V(v) \approx 0$ means "x is as convincing a picture of v as a perfect rendering", while strongly negative values flag visual disagreement (Figure 3). We also record the token-level minima of the log-likelihood and of the ratio (an error is often a single character) and the cosine distance between V's decoder states on x and on $R(v)$. Every quantity is computed with two recognisers of different design: V_1=TrOCR-base [4] (ViT encoder, BPE decoder; the printed, handwritten or scene-text checkpoint matching the domain) and V_2=PARSeq [3] (character-level permuted autoregressive decoder). Neither is fine-tuned. Each node thus carries 2×11 evidence features, seven claim-relative differences $\phi(v) - \phi(h)$, its edit type and its relative edit position.

Renderer and preprocessing (exact configuration). $R(v)$ is drawn with the Pillow library in Liberation Sans Regular, a freely licensed font metrically equivalent to Arial, chosen *a priori* as a neutral sans-serif and never tuned on any test set. The canvas height equals the crop height clamped to $[32, 96]$ pixels; the font size is $0.75\times$ that height; the text is drawn black on a white RGB canvas, vertically centred from its glyph bounding box, with a left margin of a quarter of the height and a total horizontal margin of half the height, and no blur, noise or augmentation. Real crop and rendering then pass through the *same* preprocessing of each recogniser: for TrOCR, bilinear (antialiased) resizing to 384×384 and normalisation $(x/255 - 0.5)/0.5$ in half precision; for PARSeq, bicubic resizing to 32×128 and the same normalisation. PARSeq

scores v restricted to its 94-character set and truncated to 25 characters; TrOCR scores at most 60 BPE tokens. Both are scored teacher-forced from left to right, so $s_V(v \mid \cdot)$ is a sum of per-token log-probabilities. Because the ratio subtracts two scores of the *same* string by the *same* recogniser, a font or scale mismatch shifts both terms of a hypothesis similarly; Section 6.4 measures the residual sensitivity with six alternative renderings.

3.3 The LOCUS network

Node features are standardised with statistics of the *unlabelled* pool and embedded linearly. Three relation-aware GATv2 layers [14] with edge features and residual LayerNorm update the node states:

$$\begin{aligned} h_u^{(\ell+1)} &= \mathrm{LN}\Big(h_u^{(\ell)} + \sigma\Big(\textstyle\sum_{v\in\mathcal{N}(u)} \alpha_{uv}^{(\ell)} W^{(\ell)} h_v^{(\ell)}\Big)\Big), \\ \alpha_{uv}^{(\ell)} &= \mathrm{softmax}_v\Big(a^\top \mathrm{LReLU}\big(W[h_u \| h_v \| e_{uv}]\big)\Big). \end{aligned} \tag{2}$$

Local-optimality readout. A plausibility score $q_v = w_q^\top \mathrm{MLP}(h_v)$ defines $P_{\mathrm{opt}}(v \mid G) = \exp q_v / \sum_{u\in\mathcal{V}} \exp q_u$, the probability that v is the best explanation of x within its neighbourhood. The verdict combines the claim's state, an attention pool and a max pool over its rivals, and the log-probability that the claim itself is the local optimum,

$$\mathrm{logit}\, P(y=1 \mid x,h) = \mathrm{MLP}\big([h_h \,\|\, \textstyle\sum_v a_v h_v \,\|\, \max_v h_v]\big) + \beta \log P_{\mathrm{opt}}(h \mid G), \tag{3}$$

and the same distribution gives the repair $\hat{t} = \arg\max_{v\neq h} P_{\mathrm{opt}}(v \mid G)$. We also compute *character evidence* (per-position counterfactual margins of the claim under V_1, pooled by mean/min/max); it is used only by the "LR on character evidence" baseline and by ablations, not by LOCUS.

3.4 Label-free pretraining and few-shot adaptation

On the unlabelled training images we keep those on which two OCR engines agree (a high-precision proxy for correctness) and pair each with (i) the agreed string as a pseudo-correct claim and (ii) an OCR-style corruption of it (confusable substitution, case flip, deletion or insertion) as a pseudo-incorrect claim whose repair is the original. LOCUS is trained on these graphs with $\mathcal{L} = \mathrm{BCE}\big(P(y=1 \mid x,h), \tilde{y}\big) - \lambda \log P_{\mathrm{opt}}(\tilde{t} \mid G)$ ($\lambda = 0.5$), where $\tilde{y}$ is the pseudo-label and $\tilde{t}$ the node holding the pseudo-truth; no human label is used. Given k labelled words per class, the pretrained network is fine-tuned for 300 steps (AdamW, lr $3 \cdot 10^{-4}$, batch of half the support set, class-balanced binary cross-entropy plus the same repair term on correct support words). This is repeated for five independently drawn support sets and the five fine-tuned networks are averaged (Section 4); the average is the reported LOCUS verdict, thresholded at 0.5. Two alternative heads, a logistic probe on the frozen embedding and a prior-anchored logistic head, are reported only as ablations (Table 4). Feature standardisation uses statistics of the unlabelled training pool, and no validation labels are used in the few-shot regime.

Input: word image x, engine claim h; frozen recognisers V_1, V_2; renderer R; five fine-tuned LOCUS networks.
Output: $P(\text{CORRECT} \mid x, h)$ and a repair proposal $\hat{t}$.

1. $g_1 \leftarrow V_1(x)$, $g_2 \leftarrow V_2(x)$ *(greedy readings)*
2. $\mathscr{V} \leftarrow \{h, g_1, g_2\} \cup \text{AlignedEdits}(h, g_1, g_2) \cup \text{ConfusionEdits}(h)$, truncated to $M{+}1$ nodes
3. **for** each $v \in \mathscr{V}$ and $V \in \{V_1, V_2\}$: $\rho_V(v) \leftarrow s_V(v \mid x) - s_V(v \mid R(v))$, plus token minima and latent distance
4. build typed edges (edit, sibling, path) and node features $[\phi(v) \,\|\, \phi(v) - \phi(h) \,\|$ edit type, position$]$
5. run 3 GATv2 layers; compute $P_{\text{opt}}(\cdot \mid G)$ and $\text{logit}\, P(y = 1 \mid x, h)$
6. average $P(y = 1 \mid x, h)$ over the networks fine-tuned on the five support sets
7. **return** P, and $\hat{t} = \arg\max_{v \neq h} P_{\text{opt}}(v \mid G)$ if $P < 0.5$

Algorithm 1. LOCUS inference for one claim.

Why verification does not need the truth inside the graph. Local optimality is a *necessary* condition for correctness: if $h = t$, no edit of h explains the image better, so (up to recogniser noise) no rival has $\bar{\rho}(v) > \bar{\rho}(h)$, with $\bar{\rho} = \rho_{V_1} + \rho_{V_2}$. Rejection therefore needs only *one* rival that beats the claim, not the truth. Because s_V is a sum over positions, an edit that moves an erroneous character h_i towards what is written raises $\bar{\rho}$ mainly through position i; a claim with several wrong characters thus loses to its partially corrected neighbours and to differing recogniser readings, even when the truth is several edits away and absent from $G(h)$, and a strongly negative $\rho(h)$ flags a poor fit with no rival at all. Repair, in contrast, *does* require $t \in G(h)$; Section 6.6 confirms this asymmetry.

Computational cost. For $n = |\mathscr{V}|$ hypotheses (15–21 on average), evidence extraction runs, per recogniser, one encoder pass on the crop, n on the renderings and $2n$ teacher-forced decoder passes: $O(nC_{\text{rec}})$. Graph construction manipulates strings only, and the GNN costs $O(L(|\mathscr{E}|d + nd^2)) \approx 10^6$ operations per word ($L = 3$, $d = 64$), five orders of magnitude below one ViT-base encoder pass. The cost of LOCUS is the cost of its evidence, linear in n; Section 5.6 measures it and trades it off against accuracy.

4. Experimental Setup

4.1 A verification benchmark with real OCR errors

Public OCR datasets provide images and transcriptions but no verification labels. We create them by running real OCR engines and labelling each output against the ground truth (Supplementary Figure S9a), so every INCORRECT example is a genuine engine error, not a synthetic perturbation. Five public datasets cover four domains (Table 1, Supplementary Figure S8): **FUNSD** [20], word boxes in noisy scanned forms (the Hugging Face release stores boxes on the LayoutLM 0–1000 grid; they must be rescaled to pixels before cropping); **CORD** [21], words in camera-captured receipts (official train and validation splits form the training pool); **SROIE** [22], text lines of scanned receipts, whose ground truth is entirely upper-case, so comparison is case-insensitive;

Table 1. Benchmark. Five public datasets with real OCR errors. Error rate: share of engine outputs that differ from the ground truth (exact match; SROIE case-insensitive; IIIT5K case-insensitive alphanumeric).

Dataset	domain	unit	#train	#test	avg. len.	engine	err. train (%)	err. test (%)
FUNSD	scanned forms	word	8,000	4,000	5.1	Tesseract 5	28.7	36.9
CORD	camera receipts	word	8,000	2,356	4.9	Tesseract 5	37.0	39.9
SROIE	scanned receipts	line	8,000	4,000	11.5	Tesseract 5	47.8	47.0
IAM	handwriting	word	8,000	4,000	4.2	TrOCR-small	48.4	46.4
IIIT5K	scene text	word	2,000	2,999	5.0	Tesseract 5	14.2	29.0

IAM[1], handwritten English words; and **IIIT5K**[2], scene-text words, for which we follow the standard case-insensitive alphanumeric protocol and give *every* verifier, including LOCUS and the VLM judge, the claim in this canonical form. Official splits are kept and pools are capped at 8,000 training and 4,000 test crops (fixed seed). The main engine is Tesseract 5 [23] (LSTM, `psm 7`); on handwriting Tesseract is degenerate (93% word error), so IAM verifies TrOCR-small-handwritten. A second engine, TrOCR-small, is used for engine transfer. Test error rates span 29–47%.

Data partitioning and leakage control. We keep the official splits, which are document-disjoint for FUNSD, CORD and SROIE, so all crops of a document fall in one split. Pretraining, support sets, thresholds and all learned baselines use only the *training* pool; no test crop, label or statistic enters pretraining, model selection or any threshold. A mechanical audit confirms that 0 pretraining images (2,561 FUNSD, 3,000 CORD, 2,663 SROIE, 3,000 IAM, 860 IIIT5K) come from a test split, and pixel hashing finds no train/test duplicates except 79 CORD crops from receipts duplicated across the official splits; removing them leaves LOCUS' macro-F1 (.978) and AUROC (.993) unchanged. The public IAM word release is split at word level, so it is not writer-disjoint. Hyperparameters were fixed on FUNSD and CORD, whose test sets were consulted during development, and applied unchanged to the held-out SROIE, IAM and IIIT5K.

What counts as an error. Labels inherit the datasets' conventions and noise. A share of incorrect claims differ from the ground truth only in case or punctuation (mostly fragments of neighbouring glyphs inside the word box), and our manual audit finds annotation errors and clipped crops among them (Section 7). We keep these claims (an engine that emits `Barbara |` for the field `Barbara` is wrong for a downstream consumer) but also report every method on genuine character errors only (Section 6.5). Supplementary Table S33 shows that the counterfactual graph contains the truth for 83–97% of real errors on printed and scene text and for 55% on handwriting, where many errors span several characters.

[1] IAM Handwriting Database, Research Group on Computer Vision and Artificial Intelligence, University of Bern; `https://fki.tic.heia-fr.ch/databases/iam-handwriting-database`.

[2] IIIT 5K-Word dataset, Centre for Visual Information Technology, IIIT Hyderabad; `https://cvit.iiit.ac.in/research/projects/cvit-projects/the-iiit-5k-word-dataset`.

4.2 Baselines, protocol and metrics

All baselines see the same test words, and every threshold is fitted on the same k-shot support set as LOCUS (maximising macro-F1). **Engine confidence**: Tesseract's mean word confidence or TrOCR-small's mean token probability (temperature scaling is monotone and leaves ranking metrics unchanged). **ConfidNet-style learned confidence** [7]: an MLP trained on the support set from the engine confidence and the verifier's likelihood statistics of the claim. **Re-recognition agreement**: accept iff an independent recogniser reads the same string: TrOCR-base [4], PARSeq [3] and Florence-2 [5]; for PARSeq we also report a *graded* variant ranked by its sequence confidence (agree $\rightarrow +c$, disagree $\rightarrow -c$), the strongest re-recognition baseline. **Two-engine voting**: accept iff Tesseract and TrOCR-small agree. **VLM judge**: Qwen2.5-VL-7B-Instruct [6] asked whether the image shows exactly the claimed text, scored by $P(\texttt{yes})/(P(\texttt{yes})+P(\texttt{no}))$ at the natural 0.5 threshold and at an *oracle* threshold tuned on the test set. **Strong learned baselines on our evidence**: logistic regression and gradient-boosted trees on flattened LOCUS evidence and/or character evidence, DeepSets over rivals (no edges), and LOCUS from scratch.

For each k we draw five class-balanced support sets (seeds 0–4); the test set keeps its natural class ratio, and we report the ensemble of the five models. Metrics: **macro-F1** (primary), **AUROC**, **AURC** (area under the risk–coverage curve, lower is better), **coverage@95** (share of words that can be accepted while the accepted set stays $\geq 95\%$ correct) and top-1 **repair accuracy**. In every column of a comparison table the best entry is **bold on green** and the second best underlined on blue. **Implementation:** hidden size 64, 3 GATv2 layers with 4 heads, dropout 0.2; pretraining 15 epochs (AdamW, lr 10^{-3}, batch 64); rendering in Liberation Sans; the trainable GNN has 0.05 M parameters and evidence extraction runs at $\approx$18 words/s on one RTX A6000 (Section 5.6).

Reproducibility. All numbers are produced by scripts from raw per-run result files. Code, benchmark manifests (crop coordinates, splits and claims; images and ground truth must be obtained from the dataset owners), per-run results and a step-by-step reproduction guide (Supplementary Section S8) accompany the article.

5. Results and Discussion

5.1 Main comparison

Tables 2 and 3 compare LOCUS with all baselines at $k = 25$ (other k and all metrics: Supplementary Section S6).

General-purpose verifiers are far from sufficient. Engine confidence, its learned refinement, engine voting, Florence-2 re-recognition and the 7B VLM judge all trail LOCUS by large

Table 2. Main comparison, macro-F1 at $k = 25$ labelled words per class (learned methods: ensemble of five seeds; threshold baselines: mean over the five supports). FUNSD and CORD were used for development; SROIE, IAM and IIIT5K are held out. †Threshold tuned on the test set (upper bound); ‡the best baseline chosen separately for every dataset in hindsight (both excluded from the colour marks).

Method	FUNSD	CORD	SROIE	IAM	IIIT5K	Avg.
Baselines						
Engine MSP confidence	.756	.777	.780	.802	.788	.781
ConfidNet-style learned conf. [7]	.771	.842	.826	.801	.922	.832
Two-engine voting	.609	.772	.976	.759	.862	.796
Re-OCR: TrOCR-base [4]	.605	.811	**.979**	.758	.952	.821
Re-OCR: Florence-2 [5]	.647	.615	.478	.369	.823	.586
Re-OCR: PARSeq [3]	.946	.973	.742	.879	**.993**	.907
Re-OCR: PARSeq, graded	.946	.973	.742	.879	**.993**	.907
VLM judge: Qwen2.5-VL-7B [6]	.760	.891	.866	.700	.888	.821
VLM judge, oracle threshold†	.762	.902	.883	.719	.928	.839
Best baseline per dataset, chosen in hindsight‡	.946	.973	.979	.879	.993	.954
Strong learned baselines on our evidence						
LR on flattened graph evidence	**.948**	.955	.955	.904	.969	.946
LR on character evidence	.935	.971	.959	.887	.965	.943
GBDT on all evidence	.922	.961	.962	.893	.955	.939
DeepSets over rivals (no edges)	.942	.977	.968	**.909**	.965	.952
LOCUS-GNN from scratch	.933	.969	.961	.908	.962	.947
Ours						
LOCUS zero-shot (no labels)	.918	.904	.798	.825	.968	.883
LOCUS (k-shot)	.943	**.978**	.973	.906	.971	**.954**

Table 3. Main comparison, AUROC at $k = 25$ (same protocol as Table 2).

Method	FUNSD	CORD	SROIE	IAM	IIIT5K	Avg.
Baselines						
Engine MSP confidence	.861	.892	.859	.878	.938	.886
ConfidNet-style learned conf. [7]	.845	.929	.912	.861	.980	.905
Two-engine voting	.680	.807	.976	.767	.905	.827
Re-OCR: TrOCR-base [4]	.675	.841	.978	.766	.966	.845
Re-OCR: Florence-2 [5]	.715	.685	.578	.522	.879	.676
Re-OCR: PARSeq [3]	.939	.975	.763	.885	.991	.911
Re-OCR: PARSeq, graded	**.976**	.986	.760	.914	**.995**	.926
VLM judge: Qwen2.5-VL-7B [6]	.846	.945	.941	.788	.974	.899
VLM judge, oracle threshold†	.846	.945	.941	.788	.974	.899
Best baseline per dataset, chosen in hindsight‡	.976	.986	.978	.914	.995	.970
Strong learned baselines on our evidence						
LR on flattened graph evidence	.975	**.995**	.990	.970	.991	**.984**
LR on character evidence	.973	.992	.990	.959	.993	.981
GBDT on all evidence	.969	.988	.991	.964	.990	.981
DeepSets over rivals (no edges)	.971	.994	**.992**	**.972**	.990	**.984**
LOCUS-GNN from scratch	.969	.992	.990	.969	.990	.982
Ours						
LOCUS zero-shot (no labels)	.949	.977	.966	.909	.983	.957
LOCUS (k-shot)	.974	.993	**.992**	.969	.991	**.984**

margins on every dataset. Despite 8.3B parameters the VLM judge reaches only 0.760 macro-F1 on FUNSD and 0.700 on IAM, and an oracle threshold barely helps: its errors are errors of *ranking*, not of calibration. Character-exact judgements (`O` vs. `0`, a missing comma, a case flip) are what instruction-tuned VLMs gloss over.

Re-recognition with a strong recogniser is the baseline to beat. Accepting a claim iff PARSeq reads the same string is by far the strongest baseline, and its graded variant more so. LOCUS improves on it where PARSeq is fallible: on IAM handwriting by +0.027 macro-F1 and

+0.055 AUROC, and on CORD receipts (macro-F1 0.978 vs. 0.973; AUROC gain significant, Table 9). On FUNSD the two are statistically tied, and on IIIT5K, the scene-text benchmark on which PARSeq was designed and reads almost every word correctly, PARSeq agreement remains better (0.993 vs. 0.971). On held-out SROIE the ranking of baselines flips: TrOCR's printed checkpoints were fine-tuned on SROIE receipts, so TrOCR re-recognition is strong (.979 macro-F1) while PARSeq agreement fails on these long receipt lines (.742); LOCUS is slightly below the former in macro-F1 (.973) and above it in AUROC (.992 vs. .978, Table 9). No single re-recognition baseline is the best choice on every dataset (PARSeq on forms and scene text, TrOCR on SROIE, neither on handwriting), whereas LOCUS is within 0.006 macro-F1 of the best baseline or better on four of the five datasets. Its average macro-F1 (.954) is the highest of all methods in Table 2: well above the best single baseline (.907, PARSeq), equal to choosing the best baseline separately for every dataset in hindsight (.954), and marginally above simple learners trained on our own counterfactual evidence (.939–.952).

Learned baselines on our evidence and the label-free model. Logistic regression, gradient boosting and DeepSets on the *same* counterfactual evidence are competitive with LOCUS (middle blocks): most of the gain over general-purpose baselines comes from the counterfactual, render-normalised *evidence* itself, and Section 6.3 shows that message passing adds little on top of it. Pretrained only on engine agreement and synthetic corruptions, zero-shot LOCUS already exceeds engine confidence, learned confidence, Florence-2 and the VLM judge in AUROC on every dataset, and engine voting on four of five (all except SROIE, where both voting engines include a SROIE-trained recogniser).

5.2 Label efficiency and render normalisation

Supplementary Figure S10 shows that LOCUS is close to its fully supervised accuracy with $k = 10$–25 labelled words per class on the document datasets, and keeps improving with more labels on IAM; seed variance shrinks rapidly with k (Supplementary Section S6). Supplementary Figure S7 and Table S17 isolate Equation (1): with TrOCR as the only verifier it helps where TrOCR's prior is mismatched (SROIE .956 vs. .902) and is neutral with PARSeq alone; render normalisation corrects a *biased* recogniser and is neutral for an unbiased one.

Why does removing render normalisation cost only 0.002? The average hides where the ratio acts (Supplementary Table S34). Where TrOCR's string prior is mismatched it is large: TrOCR AUROC on SROIE claims with digits rises from .901 to .976, and on lower- and mixed-case words from .714 to .807 (FUNSD), .752 to .948 (CORD) and .722 to .972 (SROIE); on upper-case words it changes nothing, and on handwriting it hurts (IAM digits .740 to .638), since a printed rendering is a poor reference there. In the full model PARSeq and the agreement features already cover the subsets where the ratio helps, hence the small ablation effect. The ratio is what makes a single, domain-mismatched recogniser usable as a verifier.

Table 4. Ablations at $k = 25$, macro-F1 (ensemble of five seeds). Δ: mean change w.r.t. the full model over the datasets where the variant was run (AUROC: Supplementary Table S8). The "LOCUS (full)" row is an independent re-run of the model of Table 2; the last two rows give the range over three further independent re-runs (run-to-run noise).

Variant	FUNSD	CORD	SROIE	IAM	IIIT5K	Δ
LOCUS (full)	.939	.979	**.974**	.902	.970	
+ CV-selected prior/anchored head	.944	.961	.965	.903	.972	-0.004
+ linear probe on embedding	.943	.970	.965	.901	.967	-0.004
w/o verifier-agreement node features	.937	.978	.963	.902	.958	-0.005
w/o edit-graph GNN (char evidence LR)	.935	.971	.959	.887	.965	-0.009
w/o label-free pretraining	.932	.969	.962	.908	.962	-0.006
w/o local-optimality term	.943	.980	.968	.899	.967	-0.002
w/o repair loss	.941	**.981**	**.974**	.899	.965	-0.001
w/o render-normalisation	.942	.977	.967	.893	.973	-0.002
w/o latent render distance	.940	.978	.971	**.914**	.972	+0.002
likelihood only (no render)	.940	.979	.969	.913	.975	+0.002
TrOCR verifier only	.942	.980	.971	.881	.975	-0.003
PARSeq verifier only	.938	.978	.969	.898	.970	-0.002
star edges only	.943	**.981**	.968	.898	.972	-0.001
no message passing	**.945**	.978	.969	.895	**.979**	+0.000
GCN backbone	.942	.978	.965	.905	.969	-0.001
GraphSAGE backbone	.933	.978	.970	.906	.966	-0.002
GIN backbone	.940	.972	.955	.889	.964	-0.009
Run-to-run noise: range over three independent re-runs						
LOCUS (full), 3 independent re-runs	.940–.941	.978–.980	.973–.973	.896–.903	.971–.972	
no message passing, 3 re-runs	.944–.945	.978–.978	.969–.970	.894–.895	.979–.979	

5.3 Ablations

Table 4 reports every component. The clearest effects are label-free pretraining and operating on the edit graph at all (versus the same character evidence without rival hypotheses). Several conceptually central components are individually *redundant* in the full model: removing message passing, the local-optimality term, the repair loss or render normalisation changes macro-F1 by less than a point on most datasets, because the two verifiers and the agreement features carry overlapping information; GCN and GraphSAGE backbones perform like GATv2 and GIN is slightly worse (-0.009). The value of LOCUS lies in *what* is compared (the claim against its counterfactual rivals, under prior-cancelled evidence) more than in the network that compares them.

Consistency of the reported numbers. The full model appears in Tables 2 and 4 and Supplementary Table S43 with slightly different values (FUNSD .943, .939, .943) because each comes from an independent run of the whole pipeline; seeds are fixed, but GPU scatter kernels are not bit-deterministic. Three further re-runs (Table 4, bottom) span at most 0.006 macro-F1 and 0.001 AUROC. Table 2 is the reference result, and effects below this noise level are not interpreted; repeated "no message passing" runs differ from the full model by -0.005 to $+0.007$ macro-F1, with changing sign.

5.4 Transfer across engines and datasets

A verifier that must be retrained for every engine is of limited use. Trained on Tesseract claims and tested on TrOCR-small claims (and vice versa), the fully supervised *ranking* transfers with

Table 5. Threshold re-calibration under engine transfer (LOCUS, macro-F1 on the target engine). err.: target error rate (%). EM: label-free prior-shift EM. 5/25: target threshold fitted on 5 or 25 labelled target words per class. In-engine: LOCUS adapted on the target engine ($k = 25$). ECE: expected calibration error at 0.5 / after EM.

Dataset	Train→Test	err.	0.5	EM	5	25	in-engine	ECE
FUNSD	Tess.→TrOCR-s	61.7	.689	.573	.817	.862	.955	.235/.359
FUNSD	TrOCR-s→Tess.	36.9	.946	.943	.907	.926	.940	.031/.028
CORD	Tess.→TrOCR-s	37.4	.728	.575	.912	.931	.957	.205/.276
CORD	TrOCR-s→Tess.	39.9	.836	.836	.870	.935	.978	.137/.137
SROIE	Tess.→TrOCR-s	6.3	.752	.625	.497	.559	.639	.036/.052
SROIE	TrOCR-s→Tess.	47.0	.767	.735	.841	.853	.973	.239/.271
IIIT5K	Tess.→TrOCR-s	28.3	.984	.983	.866	.953	.990	.015/.007
IIIT5K	TrOCR-s→Tess.	29.0	.968	.971	.886	.949	.970	.026/.020

an AUROC loss of at most 0.006 on FUNSD, CORD and IIIT5K (at most 0.023 at $k = 25$), but SROIE loses 0.02–0.07, and the 0.5 decision threshold transfers poorly everywhere because the two engines' error rates and types differ; a new engine therefore needs at least threshold re-calibration (Supplementary Table S35). The ranking transfers well across all twenty dataset pairs (AUROC .892–.995; e.g. FUNSD→CORD .995 vs. .996 in-domain, CORD→FUNSD .968 vs. .979). The weakest transfers are those *to* handwriting (SROIE→IAM .892, CORD→IAM .954) and from receipt lines to forms (SROIE→FUNSD .941). As for engines, macro-F1 at the fixed 0.5 threshold transfers much less well than the ranking (e.g. SROIE→FUNSD .620).

Re-calibrating the threshold for a new engine (Table 5). A new engine changes the error rate (FUNSD: 37% to 62%) and the error types. Label-free EM prior re-estimation with bias-corrected calibration [18] does *not* help: it lowers macro-F1 in six of eight directions and usually increases calibration error, because the class-conditional scores also shift. Re-fitting the threshold on 25 labelled target words per class recovers most of the loss where the transferred threshold fails (FUNSD .689 to .862, CORD .728 to .931 and .836 to .935, SROIE .767 to .853) but is noisier where it already works; 5 words per class are too few. Re-adapting LOCUS on 25 target words per class is best in five of eight directions and close to the best in two others (the exception, SROIE with TrOCR-s claims, has only 6.3% errors). We therefore recommend treating a new engine as a new domain: label about 25 words per class, re-adapt, and monitor accepted precision; conformal calibration on that sample would add a distribution-free guarantee [19].

5.5 Practical use: human review, repair and cost

The operational question in a digitisation pipeline (Supplementary Figure S9b) is how many words a human must check to guarantee the quality of the rest (Supplementary Figure S11, Supplementary Table S45). At 95% accepted precision, LOCUS requires reviewing 44.7% of SROIE lines (TrOCR re-recognition 45.0%, PARSeq 69.5%, the VLM judge 76.2%, engine confidence 90.9%) and 51.4% of IAM words (PARSeq 54.6%, the VLM judge 94.8%). On FUNSD, CORD and IIIT5K, LOCUS and PARSeq are within two points of each other (FUNSD

Table 6. Repair and cost. Left: share of real errors whose truth lies in $G(h)$, and top-1 exact repair accuracy (%) of substituting TrOCR's reading, PARSeq's reading, or LOCUS' label-free proposal. Right: measured throughput on 256 FUNSD words on one RTX A6000.

Dataset	errors	in $G(h)$	TrOCR	PARSeq	LOCUS
FUNSD	1474	82.8	31.3	**72.4**	67.7
CORD	941	94.9	70.4	**89.5**	81.4
SROIE	1880	92.6	**89.5**	44.1	**89.5**
IAM	1857	55.4	14.4	**46.5**	29.0
IIIT5K	869	97.0	86.8	**95.3**	93.6

Component	parameters	words/s
LOCUS evidence extraction (TrOCR-base + PARSeq, both renderings)	358M (frozen)	17.9
LOCUS GNN (trainable)	0.05M	–
PARSeq re-recognition	24M	372.3
Florence-2-base re-recognition	0.23B	14.9
Tesseract 5 (CPU, 1 thread)	–	9.3
Qwen2.5-VL-7B judge	8.3B	53.7

37.2% vs. 35.4%, CORD 37.4% vs. 38.6%, IIIT5K 25.6% vs. 25.3%), while engine confidence and the VLM judge require 1.6–2.3× more review than LOCUS on the three receipt and form datasets. At the strictest target (98%) PARSeq is better on FUNSD and IAM. Verification and repair are separate tasks, and LOCUS addresses the first. For repair (Table 6), the truth lies in the counterfactual graph for most errors, but substituting the better verifier's reading is at least as good a top-1 repair as LOCUS' label-free proposal on every dataset: LOCUS is a verifier first, and its repair head, trained only on synthetic corruptions, is a useful by-product rather than a state-of-the-art corrector. We therefore do not claim repair as a contribution. The repair loss does not affect verification either (removing it changes macro-F1 by -0.001, Table 4). In deployment the two roles combine naturally: LOCUS decides which words are sent for review, and the reviewer is shown the reading of the stronger verifier as the suggested correction. Repair is in any case bounded by graph coverage, which is analysed with multi-character errors in Section 6.6. Scoring $\sim$17 hypotheses with two recognisers and two renderings runs at about 18 words/s on one GPU, slower than a single PARSeq pass and about 3× slower than a batched 7B judge, although its frozen models are 23× smaller (Table 6); Section 5.6 analyses this cost and how to reduce it.

5.6 Computational cost, graph size and a two-stage cascade

Where the time goes. On 512 FUNSD words (one RTX A6000), graph construction runs at 3,674 words/s and the GNN at 17,878 words/s (95 edges per graph): both are negligible. The time is spent scoring $\sim$17 hypotheses with two recognisers on the crop and on each rendering ($\approx$70 recogniser passes per word); larger batches do not help (17.0 vs. 16.9 words/s for batch 32 and 128), and capping the graph at $M = 16$, 8 or 4 rivals gives 21, 32 and 49 words/s.

Graph size and neighbour selection (Table 7). Accuracy is almost flat from $M = 28$ to $M = 4$ (largest macro-F1 change 0.007), and $M = 2$ loses at most 0.012 AUROC. The recogniser readings carry most of the value: without them the share of IAM errors with the truth in the graph falls from 55.4% to 4.1%. The cap binds mainly for long SROIE lines (44.9% of claims; 7–18% elsewhere), and accuracy decreases with length (SROIE .992 for 1–3 characters to .959 for 16 or more; Supplementary Table S4). $M = 8$ is a safe choice when throughput matters.

Table 7. Graph size and composition ($k = 25$, macro-F1 / AUROC). nodes: average hypotheses per claim (FUNSD); w/s: measured words/s; cov.: share of IAM errors whose truth is in the graph (%).

Graph	nodes	w/s	FUNSD	CORD	SROIE	IAM	IIIT5K	cov.
M=2	3.0	–	.937/.962	.980/.993	.969/.991	.901/.962	.972/.991	52.0
M=4	4.9	49	.943/.969	.977/.993	.966/.991	.901/.964	.972/.991	53.9
M=8	8.6	32	.940/.964	.973/.992	.965/.991	.904/.966	.972/.991	54.6
M=16	13.7	21	.943/.972	.980/.993	.969/.992	.894/.965	.972/.990	55.1
M=28 (default)	17.2	17	.940/.974	.979/.993	.972/.992	.899/.967	.970/.991	55.4
readings only	1.9	–	.942/.960	.978/.992	.966/.990	.899/.960	.974/.990	50.8
edits only (no readings)	13.4	–	.945/.970	.976/.993	.965/.992	.899/.965	.961/.987	4.1

Table 8. Cascade ($k = 25$): accept when PARSeq reads the claim with confidence ≥ 0.95, otherwise run LOCUS. routed: share of words sent to LOCUS; early err.: error rate among early-accepted words (%); w/s: effective words/s.

	LOCUS on all words			cascade			
Dataset	F1	AUROC	routed	early err.	F1	AUROC	w/s
FUNSD	0.943	0.974	43.2	3.1	0.943	0.977	37
CORD	0.978	0.993	49.7	0.4	0.979	0.995	33
SROIE	0.973	0.992	73.0	1.9	0.972	0.988	23
IAM	0.906	0.969	68.5	1.7	0.908	0.973	24
IIIT5K	0.971	0.991	34.1	0.4	0.987	0.995	46

A two-stage cascade. Most words are easy, so we accept a claim directly when PARSeq (372 words/s) reads exactly the claim with confidence ≥ 0.95 and run LOCUS only on the rest (Table 8). Only 34–73% of words reach LOCUS, early-accepted words have 0.4–3.1% errors, macro-F1 changes by -0.001 to $+0.016$, and throughput rises to 23–46 words/s (1.3–2.6×); with $M = 8$ it would exceed 40 words/s except on SROIE. LOCUS is thus a second-stage verifier for the uncertain fraction of a stream, not a replacement for a fast reader. The cascade also closes most of the IIIT5K gap to PARSeq (.993), most of which comes from claims correct only under the case-insensitive protocol (Section 7): it reaches .987 macro-F1 and .995 AUROC, equal to the best AUROC on IIIT5K (PARSeq graded).

6. Analysis: What Drives the Gains?

6.1 Statistical significance at the document level

Words cropped from one page share scanner, font and annotator, so a word-level bootstrap overstates confidence. For FUNSD and CORD we recovered the source document of every crop (by replaying the seeded crop selection, verified against the released manifest with zero mismatches) and resample whole *documents* (50 test forms, 100 test receipts). IIIT5K has one word per image, so word-level resampling is exact; the public SROIE and IAM releases carry no document identifiers, so their intervals are word-level and may be optimistic (Table 9). Against the strongest baseline, LOCUS is significantly better on IAM in both metrics (ΔAUROC $+0.055$, ΔF1 $+0.026$, $p < 0.001$), better in AUROC on CORD at the document level ($+0.007$, $p = 0.023$; macro-F1 $+0.005$, not significant) and on SROIE ($+0.014$, $p < 0.001$, while TrOCR re-recognition is slightly better in macro-F1, -0.006, $p = 0.002$); it is statistically tied with

Table 9. Paired bootstrap (2,000 resamples) of LOCUS ($k = 25$) against the strongest baselines: difference in AUROC and in macro-F1, 95% interval and two-sided p. Resampling unit: documents for FUNSD and CORD, words otherwise.

Dataset	unit	Baseline	ΔAUROC	95% CI	p	ΔF1	95% CI	p
FUNSD	50 docs	PARSeq re-OCR (graded)	-0.002	[-0.007, +0.002]	0.252	-0.004	[-0.008, +0.001]	0.121
FUNSD	50 docs	TrOCR-base re-OCR	+0.299	[+0.266, +0.330]	<0.001	+0.338	[+0.287, +0.389]	<0.001
FUNSD	50 docs	Qwen2.5-VL-7B judge	+0.128	[+0.111, +0.146]	<0.001	+0.183	[+0.168, +0.198]	<0.001
CORD	100 docs	PARSeq re-OCR (graded)	+0.007	[+0.001, +0.015]	0.023	+0.005	[-0.001, +0.012]	0.122
CORD	100 docs	TrOCR-base re-OCR	+0.152	[+0.122, +0.184]	<0.001	+0.167	[+0.128, +0.206]	<0.001
CORD	100 docs	Qwen2.5-VL-7B judge	+0.049	[+0.036, +0.063]	<0.001	+0.088	[+0.071, +0.105]	<0.001
SROIE	4000 words	PARSeq re-OCR (graded)	+0.232	[+0.217, +0.246]	<0.001	+0.232	[+0.218, +0.246]	<0.001
SROIE	4000 words	TrOCR-base re-OCR	+0.014	[+0.010, +0.017]	<0.001	-0.006	[-0.010, -0.002]	0.002
SROIE	4000 words	Qwen2.5-VL-7B judge	+0.051	[+0.044, +0.058]	<0.001	+0.107	[+0.097, +0.118]	<0.001
IAM	4000 words	PARSeq re-OCR (graded)	+0.055	[+0.047, +0.063]	<0.001	+0.026	[+0.015, +0.037]	<0.001
IAM	4000 words	TrOCR-base re-OCR	+0.203	[+0.192, +0.214]	<0.001	+0.148	[+0.134, +0.161]	<0.001
IAM	4000 words	Qwen2.5-VL-7B judge	+0.181	[+0.169, +0.194]	<0.001	+0.206	[+0.191, +0.222]	<0.001
IIIT5K	2999 words	PARSeq re-OCR (graded)	-0.004	[-0.007, -0.001]	0.005	-0.021	[-0.028, -0.016]	<0.001
IIIT5K	2999 words	TrOCR-base re-OCR	+0.025	[+0.019, +0.031]	<0.001	+0.019	[+0.010, +0.027]	<0.001
IIIT5K	2999 words	Qwen2.5-VL-7B judge	+0.017	[+0.012, +0.023]	<0.001	+0.083	[+0.072, +0.094]	<0.001

PARSeq on FUNSD at the document level ($p = 0.25$ and $p = 0.12$) and significantly worse than PARSeq on IIIT5K (-0.004 AUROC, -0.021 macro-F1). It is significantly better than the VLM judge and than TrOCR re-recognition on every dataset except SROIE, where the latter is in-domain.

Correcting word-level intervals for correlation. SROIE and IAM carry no document identifiers, so we bound within-document correlation on FUNSD and CORD, where both resampling units exist: the ratio of document- to word-level bootstrap variance implies an intra-document correlation of at most $\hat{\rho} = 0.254$. Applying this worst case to SROIE (347 receipts) and IAM (657 writers) inflates the variance by 3.67 and 2.29 (Supplementary Table S37). All conclusions survive except that the SROIE macro-F1 deficit against TrOCR (-0.005) is no longer significant (a tie); the AUROC gain remains. **Seed level.** Compared one by one with the strongest baseline, all five $k = 25$ models gain in AUROC on IAM ($+0.030$ to $+0.046$) and SROIE, and four of five on CORD; the IAM macro-F1 gain ($+0.027$), however, holds only for the ensemble (single models -0.016 to $+0.015$; Supplementary Table S5). We therefore state the IAM advantage as one in AUROC and, for macro-F1, of the ensemble.

6.2 How much does LOCUS depend on its recognisers?

Supplementary Table S36 lists each verifier's word accuracy and LOCUS' gain over the better re-recognition baseline; Supplementary Table S42 splits every test set by *which* verifier reads the word correctly. Three regularities hold on all five datasets. (i) When both verifiers read the word correctly every method is nearly perfect. (ii) When only one verifier is right, re-recognition built on the *other* verifier fails badly, while LOCUS stays accurate because it weighs the evidence of both (IAM: 89.5% vs. 25.7% for PARSeq agreement on words only TrOCR reads, 89.0% vs. 43.6% for TrOCR agreement on words only PARSeq reads). (iii) When neither is right, all

methods degrade. LOCUS' gain is therefore governed by how often the verifiers *disagree in correctness*: it is largest on IAM, where each verifier alone is right on only 42–65% of words, and vanishes on IIIT5K, where PARSeq is right on 98.5%. LOCUS is best understood as a learned, label-efficient combination of complementary recognisers through their counterfactual evidence, not as a substitute for a strong in-domain recogniser.

6.3 Why counterfactual evidence matters more than message passing

Removing message passing does not hurt (Table 4). To understand why, Supplementary Table S43 runs five controlled variants, each pretrained label-free with the same change. Three observations explain the null effect of message passing. First, removing message passing changes neither metric by more than 0.009 on any dataset (A vs. B). Second, the *explicit* comparison of each rival with the claim does matter: removing the claim-relative features costs 0.010 to 0.023 macro-F1 on CORD, SROIE and IIIT5K (A vs. C), and adding message passing back does not recover it (C vs. D). The network is not learning the comparison from the graph; it uses the comparison when it is computed for it. Third, the graph is a depth-one star whose readout already pools over every rival, so a message-passing layer can only re-express information the readout sees anyway. Removing the rivals entirely (E) costs ranking quality on FUNSD (AUROC .974 to .961) but little elsewhere, because the claim node keeps its own evidence and its agreement with both verifiers' readings, which summarise what the strongest rivals would say. In short, what helps is counterfactual *evidence*: scoring the claim against alternatives under prior-cancelled likelihoods and making the comparison explicit; the graph is a convenient container for it, not the source of the gain.

Separating the evidence from the GNN. Table 2 decomposes the gain of LOCUS (.954 average macro-F1) over the best general-purpose baseline (PARSeq, .907). (i) Simple learners on the same counterfactual evidence and the same 25 words per class reach .939 (gradient boosting), .946 (logistic regression) and .952 (DeepSets): the evidence accounts for 70–95% of the gain. (ii) The graph network trained from scratch (.947) is no better. (iii) Label-free pretraining adds 0.007, the only consistent increment. In AUROC the simple learners equal LOCUS (.981–.984 vs. .984), and message passing contributes nothing beyond run-to-run noise. The GNN is therefore *not* necessary: a tabular or set-based classifier on the evidence is a valid, simpler implementation. We keep the graph because it holds a variable number of rivals with typed edit relations, supports pretraining with a repair objective and costs nothing at inference (Section 5.6); the scientific contribution is the evidence.

6.4 Sensitivity to the rendering

The ratio depends on a rendering $R(v)$ whose font never matches the document. We re-extracted all evidence with four other fonts (including a handwriting-like one) and two other glyph sizes on fixed subsets of FUNSD, CORD and IAM (1,000 training and 1,000 test words each) and

re-evaluated the claim's ratio alone, the training-free ρ-margin rule, and LOCUS trained on the subset (Supplementary Table S38). The trained model is insensitive to the rendering: across seven renderings its macro-F1 varies by at most 0.005 on every dataset (FUNSD .937–.939, CORD .972–.977, IAM .919–.923), and the AUROC of the claim's ratio alone by at most 0.011. The training-free rule is more sensitive (AUROC .920–.940 on FUNSD, .919–.944 on IAM); it is best with a serif or monospaced font rather than our default sans-serif, which we did not tune, and a handwriting-like font does not help on handwriting. A mismatched rendering is thus absorbed by learning; only the label-free rule would benefit from choosing the font.

Latin-script bias and other scripts. The renderer uses a Latin font and all benchmarks are Latin-script, so these results show robustness only to the choice of Latin font. The ratio itself is script-agnostic, but connected and complex scripts (Arabic, Devanagari, Bengali) need a shaping engine for rendering (e.g. HarfBuzz via Raqm) so that ligatures and conjuncts appear as in real text, a confusion model over grapheme clusters rather than code points, and script-specific recognisers, whose readings carry most of the value of the graph (Table 7). For historical documents a period-appropriate font is the natural reference. These settings are untested (Section 8).

6.5 Robustness to annotation and cropping artefacts

Supplementary Table S39 re-scores every method after removing incorrect claims that differ from the ground truth only in case or punctuation, which the audit (Section 7) shows are often annotation conventions, label noise or fragments of neighbouring glyphs. The removed share is large on FUNSD (51% of errors) and moderate on CORD, SROIE and IAM (18–23%); IIIT5K's canonical protocol already excludes such differences. LOCUS' AUROC changes by at most 0.011 (it rises slightly or is unchanged), and its ordering relative to the baselines is unchanged except that the VLM judge, whose AUROC rises most (FUNSD .846 to .928, SROIE .941 to .982), is visibly penalised by these artefacts. The conclusions of Sec. 5.1 therefore do not rest on annotation conventions.

6.6 When the truth is not in the graph, and correlated recogniser failures

Only 55.4% of IAM errors have their truth inside $G(h)$. This bounds repair, not verification: a claim is rejected when *any* rival explains the image better (Section 3). Table 10 confirms it. On IAM LOCUS rejects 90.6% of out-of-graph errors, more than the 83.7% of in-graph ones, with AUROC .971 against correct words (PARSeq .871): these are mostly multi-character errors far from the image. On printed data out-of-graph errors are harder (recall 71–81%, AUROC .920–.946); the 26 on IIIT5K are hardest (53.8%). Recall grows with edit distance (IAM 79.7%, 90.9%, 93.2% for 1, 2, $\geq$3 edits): multi-character errors are easier to verify but harder to repair. Adding up to ten TrOCR beam-search candidates raises IAM coverage only from 55.4% to 57.9% (FUNSD 82.8% to 85.1%), so coverage is limited by the recognisers, not by the graph

Table 10. Errors whose truth is inside / outside the counterfactual graph ($k = 25$). in G: share of errors with the truth in $G(h)$ (%); recall: share of these errors rejected by LOCUS or by PARSeq agreement (%); AUROC: correct words versus in-graph or out-of-graph errors. Last columns: share of errors and recall of LOCUS at edit distance 1, 2 and 3 or more.

			recall (%)			AUROC			by edit distance 1/2/≥3	
Dataset	errors	in G	LOCUS in	LOCUS out	PARSeq out	LOCUS in	LOCUS out	PARSeq out	share (%)	LOCUS recall (%)
FUNSD	1474	82.8	92.5	71.3	74.4	0.985	0.920	0.934	50.7/29.2/20.1	81.0/95.6/99.0
CORD	941	94.9	97.4	81.2	85.4	0.996	0.939	0.945	49.9/21.5/28.6	94.0/98.0/100.0
SROIE	1880	92.6	97.0	77.7	94.2	0.995	0.946	0.635	47.0/20.8/32.2	91.3/98.5/99.8
IAM	1857	55.4	83.7	90.6	94.8	0.967	0.971	0.871	43.0/27.1/29.9	79.7/90.9/93.2
IIIT5K	869	97.0	99.3	53.8	53.8	0.997	0.795	0.852	30.0/18.9/51.1	93.5/99.4/100.0

Table 11. Pseudo-label variants for label-free pretraining: precision of the pseudo-positive labels (%) | macro-F1 / AUROC of LOCUS after $k = 25$ adaptation.

Pseudo-labels	FUNSD	CORD	SROIE	IAM	IIIT5K
two-engine agreement (default)	94.5 | .942/.975	98.7 | .978/.993	99.0 | .974/.992	83.7 | .898/.967	99.3 | .971/.991
+ confidence threshold (drop lowest 30%)	96.4 | .940/.967	99.6 | .981/.993	99.2 | .970/.992	89.3 | .910/.971	99.8 | .971/.991
+ confidence weighting	96.1 | .939/.974	99.3 | .979/.993	99.3 | .973/.992	87.6 | .905/.969	99.6 | .969/.991
three-engine agreement	96.1 | .947/.967	99.5 | .977/.993	99.7 | .954/.989	97.5 | .917/.973	99.8 | .973/.992

construction.

Correlated failures of both recognisers (Supplementary Table S44). On words both verifiers misread, LOCUS is still right on 62–84% when their readings differ. When both produce the *same* wrong reading and the claim equals it, the evidence is unanimous and wrong: LOCUS rejects only 0–11.5% of such errors (FUNSD 61 words, IAM 25, IIIT5K 11; 1–4% of all errors), and re-recognition rejects none. This is the failure mode to expect on unseen engines that resemble the verifiers; verifiers from different families reduce it, and a third independent verifier and audited monitoring are the practical safeguards.

6.7 Noisy pseudo-labels and confirmation bias

Pretraining labels a word as correct when two engines agree (Tesseract and TrOCR-small; TrOCR-small and -base on IAM). Agreement precision, measured afterwards against training ground truth that is never used, ranges from 99.3% (IIIT5K) to 83.7% (IAM; Table 11), so shared engine errors could be learned as correct (confirmation bias). We tested three remedies, each pretrained and adapted exactly like LOCUS: dropping the 30% least confident pseudo-positives, weighting them by confidence rank, and requiring PARSeq to agree as well. All raise IAM precision (to 89.3%, 87.6% and 97.5%), and accuracy follows precision: IAM macro-F1 rises from .898 to .910, .905 and .917 (AUROC .967 to .973 with three engines). Where agreement is already 94–99% precise the remedies change macro-F1 by at most 0.005, except that the PARSeq veto lowers SROIE to .954, probably because PARSeq often misreads SROIE lines. Confirmation bias is thus real but small after k-shot adaptation, and confidence thresholding is a cheap default (it never cost more than 0.008 AUROC).

7. Qualitative Analysis and Manual Audit

Qualitative examples, drawn by a seeded random choice inside rule-defined strata and showing LOCUS' failures as often as its successes, are given in Supplementary Figures S13 and S12.

Regimes. Supplementary Table S40 counts the two regimes that matter on the full test sets. Correct claims that PARSeq misreads are frequent on every dataset, and LOCUS accepts most of them (Supplementary Figure S12); this is the regime in which comparing hypotheses under two verifiers pays off. Character errors that PARSeq reproduces are rarer and LOCUS rejects only a small fraction of them (Supplementary Figure S5).

Manual audit. To understand the errors behind these numbers, we inspected two random samples of up to 20 test words per dataset: (i) words on which LOCUS decides wrongly and (ii) incorrect claims that PARSeq agrees with. Each crop was labelled *genuine* (the ground truth matches the visible text), *label noise*, *crop problem* (text clipped or a neighbouring glyph inside the box) or *ambiguous* (Supplementary Table S41). Two conclusions follow. First, fewer than half of LOCUS' apparent mistakes are genuine verification errors on a correct label (42 of 100 sampled): 25 are label noise, 9 crop problems and 8 ambiguous, and on IIIT5K 16 of 20 are claims that count as correct only after the case-insensitive, punctuation-free normalisation of the protocol (*e.g.* `HoLLywooD` for an upper-case sign), which LOCUS, judging the exact appearance of the string, rejects; this protocol mismatch, rather than a visual error, explains most of LOCUS' gap to PARSeq agreement (which compares normalised strings) on IIIT5K. Second, the stratum "errors that PARSeq agrees with" consists mostly of cases where the claim matches what is visible: only 10 of 92 sampled words are genuine errors, against 46 label-noise and 21 crop problems, so low rejection rates there are not a weakness of either verifier. The genuine failures that remain fall into two identifiable modes: rejecting correct short or case-ambiguous claims when PARSeq reads another case (CORD: `Rp`, `x`, `Pop`), and accepting capitalisation errors in handwriting (IAM: `pinkie`, `market`, `church`). The audit was performed by a single annotator; the annotated sheets are released with the code.

8. Discussion and Limitations

When does comparing hypotheses help? LOCUS helps when its two recognisers *disagree in correctness*: on words only one verifier reads correctly, re-recognition with the other is right 26–62% of the time and LOCUS 64–99% (Supplementary Table S42). It gains most where recognisers are fallible and complementary (handwriting, camera receipts), least where one in-domain recogniser is nearly always right, and it does not require knowing in advance *which* recogniser to trust.

Limitations. All design decisions were made on FUNSD and CORD; SROIE, IAM and

IIIT5K are held out. (i) Repair is bounded by graph coverage (about half of the handwriting errors); verification is not (Section 6.6). (ii) Pretraining relies on engine agreement (84–99% precise); stricter consensus helps where it is imprecise (Section 6.7). (iii) TrOCR's printed checkpoints were fine-tuned on SROIE and PARSeq was developed on IIIT5K; the re-recognition baselines use the same recognisers. (iv) Message passing, the local-optimality term and render normalisation are individually redundant in the full model: the evidence, not the network, is the contribution. (v) Under protocols that normalise case and punctuation (IIIT5K) a claim such as `HoLLywooD` counts as correct but looks wrong. (vi) All datasets are Latin-script; connected and complex scripts and historical documents need a shaping renderer, grapheme-level confusions and script-specific recognisers and are untested. (vii) Evidence extraction runs at about 18 words/s; pruning and a cascade reach roughly 40 words/s, which suits the uncertain fraction of a production stream, not exhaustive industrial-scale verification. (viii) When both recognisers make the same error and the claim agrees, LOCUS almost never detects it. (ix) A new engine needs about 25 labelled words per class; label-free recalibration did not help. (x) The IAM macro-F1 gain holds for the ensemble but not every single model, and IAM is not writer-disjoint. Deployments should monitor accepted precision on audited samples.

9. Conclusion

We reframed OCR verification as asking whether a claim is the locally optimal explanation of its image among OCR-plausible counterfactual alternatives, and made the comparison visual with a render-normalised likelihood ratio under two frozen recognisers. LOCUS, a relation-aware graph network over counterfactual string graphs, is pretrained without human labels and adapts from a handful of labelled words. On a new benchmark of real OCR errors over five public datasets it outperforms all general-purpose verifiers in AUROC on every dataset and, against re-recognition with state-of-the-art recognisers, is the only verifier close to the best everywhere. Our analyses locate the benefit in the counterfactual evidence and label-free pretraining rather than in deep message passing: simple learners on the same evidence come within 0.015 macro-F1, so the graph network is a convenient rather than a necessary implementation. Future work will extend the confusion model and verifiers to non-Latin scripts and make the evidence aware of each dataset's comparison protocol.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Ethical statement. This study uses only publicly available datasets (FUNSD, CORD, SROIE, IAM and IIIT5K) under their respective licences and publicly released pretrained models. It

involves no human participants, animals or personal data beyond what the public datasets contain, and therefore required no ethical approval.

Conflicts of interest / competing interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability. The benchmark manifests, all code, raw result files and the annotated audit sheets, together with a step-by-step reproduction guide and the Supplementary Material, are available at `https://github.com/Shikhar2004-prog/LOCUS-OCR-Verification`.

Prior publication. This article is an extended version of the conference paper by Saxena *et al.* [13], which verified OCR with character-level relational line graphs on one private dataset. The present article introduces a new segmentation-free model (counterfactual string graphs with render-normalised evidence and label-free pretraining), five public benchmarks with two OCR engines, new baselines and all analyses; no text, table or figure is reused from the conference version.

Author contributions. Shikhar Dubey and Sanhita Saxena contributed equally to conceptualisation, methodology, software, experiments, analysis and writing.

Acknowledgement. The authors thank the creators of the FUNSD, CORD, SROIE, IAM and IIIT5K datasets and of the open-source TrOCR, PARSeq, Florence-2, Qwen2.5-VL and Tesseract models used in this work.

References

[1] Fu, L., Kuang, Z., Song, J., Huang, M., Yang, B., Li, Y., Zhu, L., Luo, Q., Wang, X., Lu, H., Li, Z., Tang, G., Shan, B., Lin, C., Liu, Q., Wu, B., Feng, H., Liu, H., Huang, C., . . . Bai, X. (2025). OCRBench v2: An improved benchmark for evaluating large multimodal models on visual text localization and reasoning. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

[2] He, Z., Zhang, C., Wu, Z., Chen, Z., Zhan, Y., Li, Y., Zhang, Z., Wang, X., & Qiu, M. (2025). Seeing is believing? Mitigating OCR hallucinations in multimodal large language models. In *Advances in Neural Information Processing Systems*.

[3] Bautista, D., & Atienza, R. (2022). Scene text recognition with permuted autoregressive sequence models. In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 178–196). Springer.

[4] Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., & Wei, F. (2023). TrOCR: Transformer-based optical character recognition with pre-trained models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, *37*(11), 13094–13102.

[5] Xiao, B., Wu, H., Xu, W., Dai, X., Hu, H., Lu, Y., Zeng, M., Liu, C., & Yuan, L. (2024). Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4818–4829).

[6] Qwen Team. (2025). *Qwen2.5-VL-7B-Instruct* [Vision-language model]. Alibaba Cloud. `https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct`

[7] Corbière, C., Thome, N., Saporta, A., Vu, T.-H., Cord, M., & Pérez, P. (2022). Confidence estimation via auxiliary models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *44*(10), 6043–6055.

[8] Kanerva, J., Ledins, C., Käpyaho, S., & Ginter, F. (2025). OCR error post-correction with LLMs in historical documents: No free lunches. In *Proceedings of the Third Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL)*.

[9] Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., & Park, S. (2022). OCR-free document understanding transformer. In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 498–517). Springer.

[10] Poznanski, J., Lo, K., & Soldaini, L. (2026). The olmOCR project: Building fully open OCR using VLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations* (pp. 626–635). Association for Computational Linguistics.

[11] Leng, S., Zhang, H., Chen, G., Li, X., Lu, S., Miao, C., & Bing, L. (2024). Mitigating object hallucinations in large vision-language models through visual contrastive decoding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 13872–13882).

[12] Dubey, S., Mittal, K., Behera, S. K., Ravikiran, M., Kumar, N., Shigwan, S., & Saluja, R. (2025). Multi-feature graph convolution network for Hindi OCR verification. In *Proceedings of the 1st Workshop on Benchmarks, Harmonization, Annotation, and Standardization for Human-Centric AI in Indian Languages (BHASHA)* (pp. 1–10). Association for Computational Linguistics.

[13] Saxena, S., Tripathi, V., & Abbas, S. (2026). Reading between the characters: Label-efficient OCR verification with graph neural networks. In *Proceedings of NICCAI 2026*, Hyderabad, India (to appear).

[14] Brody, S., Alon, U., & Yahav, E. (2022). How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*.

[15] Arazo, E., Ortego, D., Albert, P., O'Connor, N. E., & McGuinness, K. (2020). Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *International Joint Conference on Neural Networks (IJCNN)* (pp. 1–8). IEEE.

[16] Sohn, K., Berthelot, D., Carlini, N., Zhang, Z., Zhang, H., Raffel, C. A., Cubuk, E. D., Kurakin, A., & Li, C.-L. (2020). FixMatch: Simplifying semi-supervised learning with consistency and confidence. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 596–608).

[17] Minderer, M., Djolonga, J., Romijnders, R., Hubis, F., Zhai, X., Houlsby, N., Tran, D., & Lucic, M. (2021). Revisiting the calibration of modern neural networks. In *Advances in Neural Information Processing Systems* (Vol. 34, pp. 15682–15694).

[18] Alexandari, A., Kundaje, A., & Shrikumar, A. (2020). Maximum likelihood with bias-corrected calibration is hard-to-beat at label shift adaptation. In *Proceedings of the 37th International Conference on Machine Learning (ICML)* (pp. 222–232). PMLR.

[19] Angelopoulos, A. N., & Bates, S. (2023). Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, *16*(4), 494–591.

[20] Jaume, G., Ekenel, H. K., & Thiran, J.-P. (2019). FUNSD: A dataset for form understanding in noisy scanned documents. In *International Conference on Document Analysis and Recognition Workshops (ICDARW)* (Vol. 2, pp. 1–6). IEEE.

[21] Park, S., Shin, S., Lee, B., Lee, J., Surh, J., Seo, M., & Lee, H. (2019). CORD: A consolidated receipt dataset for post-OCR parsing. In *Workshop on Document Intelligence at NeurIPS 2019*.

[22] Huang, Z., Chen, K., He, J., Bai, X., Karatzas, D., Lu, S., & Jawahar, C. V. (2019). ICDAR2019 competition on scanned receipt OCR and information extraction. In *International Conference on Document Analysis and Recognition (ICDAR)* (pp. 1516–1520). IEEE.

[23] Tesseract OCR contributors. (2021). *Tesseract OCR* (Version 5) [Computer software]. `https://github.`

com/tesseract-ocr/tesseract

Author's biography



Shikhar Dubey is a researcher in the Intelligent Systems group of the School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, Himachal Pradesh, India. His research addresses the reliability of optical character recognition, in particular OCR verification with graph neural networks and unsupervised methods for low-resolution text. He has published work on multi-feature graph convolution networks for Hindi OCR verification at the BHASHA workshop (2025) and on unsupervised confirmation of low-resolution OCR at the Mind, Brain, and Consciousness Conference (2025). ORCID: 0009-0001-9035-4392. E-mail: s24130@students.iitmandi.ac.in.



Sanhita Saxena is an M.Tech Research Scholar in the School of Mathematical and Statistical Sciences, Indian Institute of Technology Mandi, Himachal Pradesh, India. Her research focuses on label-efficient verification of optical character recognition with graph neural networks: she is the lead author of "Reading Between the Characters: Label-Efficient OCR Verification with Graph Neural Networks" (NICCAI 2026, Hyderabad), which models each word as a relational line graph of real-versus-rendered character comparisons and reaches near fully supervised accuracy from 25 labelled examples per class. Her interests include few-shot learning, graph representation learning and document image analysis. Before her research degree she trained as a full-stack (MERN) web developer. E-mail: s25008@students.iitmandi.ac.in.